Design patterns

1. Creational Patterns

These deal with object creation, ensuring objects are created in a manner suitable for the situation.

- **Singleton**: Ensures a class has only one instance and provides a global point of access to it.
- **Factory Method**: Defines an interface for creating objects but lets subclasses alter the type of objects created.
- **Abstract Factory**: Provides an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Builder**: Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.
- **Prototype**: Creates new objects by copying an existing object, the prototype, instead of creating new instances.

2. Structural Patterns

These focus on how classes and objects are composed to form larger structures.

- **Adapter**: Allows incompatible interfaces to work together by converting the interface of a class into another expected by the client.
- **Bridge**: Decouples an abstraction from its implementation so the two can vary independently.
- **Composite**: Composes objects into tree-like structures to represent part-whole hierarchies, allowing individual objects and compositions to be treated uniformly.
- **Decorator**: Attaches additional responsibilities to an object dynamically, without altering its structure.
- **Facade**: Provides a simplified interface to a complex subsystem.
- **Flyweight**: Reduces the cost of creating and using a large number of similar objects by sharing as much data as possible.
- **Proxy**: Provides a surrogate or placeholder for another object to control access to it.

3. Behavioral Patterns

These deal with object collaboration and delegation of responsibility.

- **Chain of Responsibility**: Passes a request along a chain of handlers until one handles it.
- **Command**: Encapsulates a request as an object, thereby allowing for parameterization of clients with different requests and support for undoable operations.
- **Interpreter**: Defines a grammar for a language and uses an interpreter to interpret the sentences in the language.

- **Iterator**: Provides a way to access elements of a collection sequentially without exposing its underlying representation.
- **Mediator**: Centralizes complex communications and control logic between related objects by encapsulating interactions in a mediator object.
- **Memento**: Allows the capture and restoration of an object's state without violating encapsulation.
- **Observer**: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated.
- **State**: Allows an object to alter its behavior when its internal state changes.
- **Strategy**: Defines a family of algorithms, encapsulates each one, and makes them interchangeable based on context.
- **Template Method**: Defines the skeleton of an algorithm in a method, deferring some steps to subclasses.
- **Visitor**: Represents an operation to be performed on elements of an object structure, allowing new operations to be added without modifying the structure.

These patterns help structure and optimize code for reuse, flexibility, and scalability in various scenarios.

Singleton Pattern

Description:

The **Singleton Pattern** ensures that a class has only **one instance** and provides a global point of access to it. This is useful when exactly one object is needed to coordinate actions across the system (e.g., managing a database connection, configuration settings, or logging).

Key Concepts:

- **Private constructor**: Prevents the direct creation of instances from outside the class.
- **Static method**: Provides the only way to get the instance.
- **Lazy initialization** (optional): The instance is created when it's requested for the first time.

Example:

A simple example of a Singleton in Java might be a logging service.

java
Skopiuj kod
```
public class Logger {
    // The single instance of the class (eager initialization)
    private static final Logger instance = new Logger();
```

```
    // Private constructor to prevent instantiation
    private Logger() {}

    // Public method to provide access to the single instance
    public static Logger getInstance() {
        return instance;
    }

    // Logging method
    public void log(String message) {
        System.out.println("Log message: " + message);
    }
}

// Usage example
public class Main {
    public static void main(String[] args) {
        Logger logger = Logger.getInstance();
        logger.log("This is a singleton logger message.");
    }
}
```

Explanation:

1. **Private Constructor**: The constructor is private, so no other classes can instantiate the `Logger` class directly.
2. **Static Instance**: The class holds a `static final` instance of itself, ensuring only one instance exists.
3. **Global Access**: The `getInstance()` method allows global access to the single instance.
4. **Logging Functionality**: The `log()` method provides the functionality of the singleton class, which can be accessed globally.

In this example, the logger can be used across the application, and only one instance of the `Logger` is ever created.

4o

Powiedziałeś(-aś):
describe and give a code example for Factory method

ChatGPT powiedział:
ChatGPT

Factory Method Pattern

Description:

The **Factory Method Pattern** defines an interface for creating objects, but allows subclasses to alter the type of objects that will be created. Instead of directly instantiating objects using `new`, the factory method delegates the task of object creation to subclasses, thus making the code more flexible and reusable.

Key Concepts:

- **Interface or Abstract Class**: Declares the factory method that will return an instance of a product.
- **Concrete Subclasses**: Implement the factory method to create and return specific product objects.
- **Product Interface**: Defines the common interface for the objects that the factory method will create.

Example:

Let's say we have a transportation system where we want to create different types of vehicles (like `Car` and `Bike`), but the client code shouldn't directly instantiate these classes.

java
Skopiuj kod
```java
// Product interface
interface Vehicle {
    void drive();
}

// Concrete Product 1
class Car implements Vehicle {
    @Override
    public void drive() {
        System.out.println("Driving a car...");
    }
}

// Concrete Product 2
class Bike implements Vehicle {
    @Override
    public void drive() {
        System.out.println("Riding a bike...");
    }
}
```

```java
// Creator class with Factory Method
abstract class VehicleFactory {
    // Factory method
    public abstract Vehicle createVehicle();

    // Method using the created product
    public void testDrive() {
        Vehicle vehicle = createVehicle();
        vehicle.drive();
    }
}

// Concrete Factory 1
class CarFactory extends VehicleFactory {
    @Override
    public Vehicle createVehicle() {
        return new Car(); // Returns a car
    }
}

// Concrete Factory 2
class BikeFactory extends VehicleFactory {
    @Override
    public Vehicle createVehicle() {
        return new Bike(); // Returns a bike
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        VehicleFactory carFactory = new CarFactory();
        carFactory.testDrive(); // Output: Driving a car...

        VehicleFactory bikeFactory = new BikeFactory();
        bikeFactory.testDrive(); // Output: Riding a bike...
    }
}
```

Explanation:

1. **Vehicle Interface**: Defines the common interface (`Vehicle`) for the products (`Car` and `Bike`) that the factory method will create.
2. **Concrete Product Classes**: `Car` and `Bike` implement the `Vehicle` interface and provide their specific behavior in the `drive()` method.
3. **Abstract Factory (Creator)**: `VehicleFactory` declares the factory method `createVehicle()` to be implemented by subclasses, which will return the product.
4. **Concrete Factories**: `CarFactory` and `BikeFactory` override the factory method and return specific product instances (`Car` or `Bike`).
5. **Client Code**: The client calls `testDrive()` without needing to know the specific class being instantiated (either `Car` or `Bike`), making the code more flexible.

This design allows for easy extension. If a new vehicle type is added, you can simply create a new factory class without modifying existing code.

Abstract Factory Pattern

Description:

The **Abstract Factory Pattern** provides an interface for creating families of related or dependent objects without specifying their concrete classes. This pattern is useful when a system needs to be independent of how its objects are created, and when multiple families of products are required (e.g., creating GUI components for different platforms like Windows, macOS).

Key Concepts:

- **Abstract Factory Interface**: Declares methods for creating abstract products.
- **Concrete Factories**: Implement the abstract factory interface and create concrete products.
- **Product Interfaces**: Define a family of related products.
- **Concrete Products**: Implement the product interfaces.

Example:

Let's consider a UI toolkit for two different operating systems, **Windows** and **MacOS**. The UI toolkit creates buttons and checkboxes. The client doesn't need to know the specific OS; it only interacts with the abstract interfaces.

java
Skopiuj kod
```
// Abstract Product 1
interface Button {
    void paint();
}

// Abstract Product 2
```

```
interface Checkbox {
    void render();
}

// Concrete Product 1 - Windows Button
class WindowsButton implements Button {
    @Override
    public void paint() {
        System.out.println("Rendering a button in Windows style.");
    }
}

// Concrete Product 2 - Windows Checkbox
class WindowsCheckbox implements Checkbox {
    @Override
    public void render() {
        System.out.println("Rendering a checkbox in Windows
style.");
    }
}

// Concrete Product 1 - MacOS Button
class MacOSButton implements Button {
    @Override
    public void paint() {
        System.out.println("Rendering a button in MacOS style.");
    }
}

// Concrete Product 2 - MacOS Checkbox
class MacOSCheckbox implements Checkbox {
    @Override
    public void render() {
        System.out.println("Rendering a checkbox in MacOS style.");
    }
}

// Abstract Factory
interface UIFactory {
    Button createButton();
    Checkbox createCheckbox();
```

```
}

// Concrete Factory 1 - Windows Factory
class WindowsFactory implements UIFactory {
    @Override
    public Button createButton() {
        return new WindowsButton(); // Creates a Windows button
    }

    @Override
    public Checkbox createCheckbox() {
        return new WindowsCheckbox(); // Creates a Windows checkbox
    }
}

// Concrete Factory 2 - MacOS Factory
class MacOSFactory implements UIFactory {
    @Override
    public Button createButton() {
        return new MacOSButton(); // Creates a MacOS button
    }

    @Override
    public Checkbox createCheckbox() {
        return new MacOSCheckbox(); // Creates a MacOS checkbox
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        // Example for Windows UI
        UIFactory windowsFactory = new WindowsFactory();
        Button windowsButton = windowsFactory.createButton();
        Checkbox windowsCheckbox = windowsFactory.createCheckbox();
        windowsButton.paint();        // Output: Rendering a button
in Windows style.
        windowsCheckbox.render();     // Output: Rendering a
checkbox in Windows style.

        // Example for MacOS UI
```

```
        UIFactory macFactory = new MacOSFactory();
        Button macButton = macFactory.createButton();
        Checkbox macCheckbox = macFactory.createCheckbox();
        macButton.paint();            // Output: Rendering a button
in MacOS style.
        macCheckbox.render();         // Output: Rendering a
checkbox in MacOS style.
    }
}
```

Explanation:

1. **Abstract Products**: `Button` and `Checkbox` are the abstract product interfaces that define the operations (`paint()` and `render()`).
2. **Concrete Products**: `WindowsButton`, `WindowsCheckbox`, `MacOSButton`, and `MacOSCheckbox` are specific implementations of these abstract products, representing different variations of UI components for Windows and MacOS.
3. **Abstract Factory**: `UIFactory` is the abstract factory interface that declares methods for creating a family of related products (buttons and checkboxes).
4. **Concrete Factories**: `WindowsFactory` and `MacOSFactory` implement the `UIFactory` interface to create concrete products (Windows or MacOS specific UI components).
5. **Client Code**: The client code uses the abstract factory interface, creating buttons and checkboxes without knowing the specific details of which OS they belong to.

This pattern helps create families of related objects (e.g., GUI components for different platforms) while abstracting the implementation details. It ensures that the related products are used together and makes it easy to add support for a new family of products without changing the client code.

4o

Powiedziałeś(-aś):
describe and give a code example for Bridge

ChatGPT powiedział:
ChatGPT

Bridge Pattern

Description:

The **Bridge Pattern** decouples an abstraction from its implementation, allowing the two to evolve independently. It achieves this by creating two separate hierarchies: one for the

abstraction and one for the implementation. This pattern is useful when both the abstraction and implementation need to be extended without affecting each other.

Key Concepts:

- **Abstraction**: Defines the high-level control structure and uses a reference to the implementor to delegate work.
- **Implementor**: Defines the interface for the implementation part that the abstraction uses.
- **Concrete Abstraction**: Extends the abstraction with additional functionality.
- **Concrete Implementor**: Provides the concrete implementation of the implementor interface.

Example:

Imagine a scenario where we have different shapes (like circles and squares) and each shape can be drawn in different rendering styles (e.g., vector and raster rendering). The shape and rendering style should be independent, so we use the Bridge pattern.

java
Skopiuj kod
```
// Implementor interface
interface Renderer {
    void renderShape(String shapeType);
}

// Concrete Implementor 1 - Vector Rendering
class VectorRenderer implements Renderer {
    @Override
    public void renderShape(String shapeType) {
        System.out.println("Rendering " + shapeType + " as lines and
points (vector).");
    }
}

// Concrete Implementor 2 - Raster Rendering
class RasterRenderer implements Renderer {
    @Override
    public void renderShape(String shapeType) {
        System.out.println("Rendering " + shapeType + " as pixels
(raster).");
    }
}

// Abstraction
```

```
abstract class Shape {
    protected Renderer renderer;

    // Constructor takes a renderer object
    public Shape(Renderer renderer) {
        this.renderer = renderer;
    }

    public abstract void draw(); // Abstract method to be
implemented by concrete shapes
}

// Concrete Abstraction - Circle
class Circle extends Shape {
    private int radius;

    public Circle(Renderer renderer, int radius) {
        super(renderer);
        this.radius = radius;
    }

    @Override
    public void draw() {
        renderer.renderShape("Circle with radius " + radius);
    }
}

// Concrete Abstraction - Square
class Square extends Shape {
    private int side;

    public Square(Renderer renderer, int side) {
        super(renderer);
        this.side = side;
    }

    @Override
    public void draw() {
        renderer.renderShape("Square with side length " + side);
    }
}
```

```
// Client code
public class Main {
    public static void main(String[] args) {
        // Drawing a circle using vector rendering
        Renderer vectorRenderer = new VectorRenderer();
        Shape vectorCircle = new Circle(vectorRenderer, 5);
        vectorCircle.draw(); // Output: Rendering Circle with radius
5 as lines and points (vector).

        // Drawing a square using raster rendering
        Renderer rasterRenderer = new RasterRenderer();
        Shape rasterSquare = new Square(rasterRenderer, 10);
        rasterSquare.draw(); // Output: Rendering Square with side
length 10 as pixels (raster).
    }
}
```

Explanation:

1. **Implementor**: The `Renderer` interface defines the method `renderShape()` for rendering shapes. `VectorRenderer` and `RasterRenderer` are the concrete implementors that provide specific implementations for rendering shapes as vectors or rasters.
2. **Abstraction**: The `Shape` abstract class uses a `Renderer` object to delegate the rendering work. This decouples the shape itself from the rendering mechanism.
3. **Concrete Abstractions**: `Circle` and `Square` are concrete implementations of `Shape`. They extend the `Shape` class and specify how the shapes will be drawn while delegating the rendering details to the `Renderer`.
4. **Client Code**: The client can mix and match different shapes (`Circle`, `Square`) with different renderers (`VectorRenderer`, `RasterRenderer`). By using the Bridge pattern, the abstraction (shape) and implementation (renderer) can vary independently.

This pattern is particularly useful when the number of possible combinations of abstraction and implementation classes is high, allowing for cleaner, more maintainable code.

4o

Powiedziałeś(-aś):
describe and give a code example for Composite

ChatGPT powiedział:
ChatGPT

Composite Pattern

Description:

The **Composite Pattern** is used to treat individual objects and compositions of objects uniformly. It allows you to build tree structures where individual objects and groups of objects are treated the same way. This is particularly useful when dealing with hierarchical structures like a file system, where files and folders can both be treated as "components."

Key Concepts:

- **Component**: The common interface for both individual objects (leaf) and composite objects.
- **Leaf**: Represents individual objects in the composition that do not have any children.
- **Composite**: Represents complex components (containers) that may contain leaf objects or other composites.
- **Client**: Interacts with objects through the common component interface, treating both leaves and composites uniformly.

Example:

Let's take an example of a **file system** where both **files** and **directories** are components, and directories can contain both files and other directories.

java
Skopiuj kod
```java
// Component interface
interface FileSystemComponent {
    void showDetails();
}

// Leaf - File
class File implements FileSystemComponent {
    private String name;

    public File(String name) {
        this.name = name;
    }

    @Override
    public void showDetails() {
        System.out.println("File: " + name);
    }
}

// Composite - Directory
```

```
class Directory implements FileSystemComponent {
    private String name;
    private List<FileSystemComponent> components = new
ArrayList<>();

    public Directory(String name) {
        this.name = name;
    }

    // Add component (file or directory) to the directory
    public void addComponent(FileSystemComponent component) {
        components.add(component);
    }

    @Override
    public void showDetails() {
        System.out.println("Directory: " + name);
        for (FileSystemComponent component : components) {
            component.showDetails(); // Delegates the call to leaf
or composite children
        }
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        // Create some files
        File file1 = new File("file1.txt");
        File file2 = new File("file2.jpg");
        File file3 = new File("file3.docx");

        // Create a directory and add files
        Directory directory1 = new Directory("Documents");
        directory1.addComponent(file1);
        directory1.addComponent(file2);

        // Create another directory and add a file
        Directory directory2 = new Directory("Projects");
        directory2.addComponent(file3);
```

```
        // Create a root directory and add both directories
        Directory rootDirectory = new Directory("Root");
        rootDirectory.addComponent(directory1);
        rootDirectory.addComponent(directory2);

        // Display the structure
        rootDirectory.showDetails();
    }
}
```

Explanation:

1. **Component Interface**: The `FileSystemComponent` interface declares the common method `showDetails()` for both `File` (leaf) and `Directory` (composite).
2. **Leaf (File)**: The `File` class implements `FileSystemComponent` and represents a leaf node in the tree (i.e., an object without children).
3. **Composite (Directory)**: The `Directory` class also implements `FileSystemComponent` but can contain other `FileSystemComponent` objects, allowing for tree structures.
4. **Client Code**: The client can interact with both `File` and `Directory` through the common `FileSystemComponent` interface. The `showDetails()` method works recursively, treating both individual files and directories uniformly.

Usage Example Output:

mathematica
Skopiuj kod

```
Directory: Root
Directory: Documents
File: file1.txt
File: file2.jpg
Directory: Projects
File: file3.docx
```

This pattern is particularly useful in scenarios involving hierarchical data, such as file systems, organizational structures, or graphical user interfaces, where individual items and groups of items need to be treated similarly.

4o

Powiedziałeś(-aś):
describe and give a code example for Decorator

ChatGPT powiedział:
ChatGPT

Decorator Pattern

Description:

The **Decorator Pattern** allows behavior to be added to individual objects, dynamically, without affecting the behavior of other objects from the same class. It provides a flexible alternative to subclassing for extending functionality. The decorator wraps an object to modify or extend its behavior while preserving the original object's interface.

Key Concepts:

- **Component**: The original object interface.
- **Concrete Component**: The object being decorated.
- **Decorator**: The abstract class or interface that wraps a component.
- **Concrete Decorators**: Extends the functionality of the component by adding behavior before or after calling the method of the component.

Example:

Let's consider a simple example of a **coffee shop** where you can order a plain coffee and dynamically add different condiments (like milk, sugar, or vanilla) using the decorator pattern.

java
Skopiuj kod
```java
// Component interface
interface Coffee {
    String getDescription();
    double cost();
}

// Concrete Component - Plain Coffee
class PlainCoffee implements Coffee {
    @Override
    public String getDescription() {
        return "Plain Coffee";
    }

    @Override
    public double cost() {
        return 2.0;
    }
}
```

```java
// Decorator - Base Decorator class
abstract class CoffeeDecorator implements Coffee {
    protected Coffee decoratedCoffee;

    public CoffeeDecorator(Coffee coffee) {
        this.decoratedCoffee = coffee;
    }

    @Override
    public String getDescription() {
        return decoratedCoffee.getDescription();
    }

    @Override
    public double cost() {
        return decoratedCoffee.cost();
    }
}

// Concrete Decorator 1 - Adding Milk
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return decoratedCoffee.getDescription() + ", Milk";
    }

    @Override
    public double cost() {
        return decoratedCoffee.cost() + 0.5;
    }
}

// Concrete Decorator 2 - Adding Sugar
class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }
```

```
    @Override
    public String getDescription() {
        return decoratedCoffee.getDescription() + ", Sugar";
    }

    @Override
    public double cost() {
        return decoratedCoffee.cost() + 0.2;
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        // Order a plain coffee
        Coffee coffee = new PlainCoffee();
        System.out.println(coffee.getDescription() + " $" +
coffee.cost()); // Output: Plain Coffee $2.0

        // Add milk to the coffee
        coffee = new MilkDecorator(coffee);
        System.out.println(coffee.getDescription() + " $" +
coffee.cost()); // Output: Plain Coffee, Milk $2.5

        // Add sugar to the coffee
        coffee = new SugarDecorator(coffee);
        System.out.println(coffee.getDescription() + " $" +
coffee.cost()); // Output: Plain Coffee, Milk, Sugar $2.7
    }
}
```

Explanation:

1. **Component Interface**: The `Coffee` interface defines the basic structure of a coffee (`getDescription()` and `cost()` methods).
2. **Concrete Component**: `PlainCoffee` is a concrete class that implements the `Coffee` interface, representing a basic, undecorated coffee.
3. **Decorator Class**: `CoffeeDecorator` is an abstract class that implements the `Coffee` interface and holds a reference to a `Coffee` object, delegating its methods to the underlying `Coffee` object.

4. **Concrete Decorators**: `MilkDecorator` and `SugarDecorator` are concrete decorator classes that extend `CoffeeDecorator`, adding specific functionality (i.e., adding milk or sugar) to the coffee.
5. **Client Code**: The client can dynamically wrap coffee objects with any combination of decorators (e.g., adding milk, then sugar).

Usage Example Output:

bash
Skopiuj kod
```
Plain Coffee $2.0
Plain Coffee, Milk $2.5
Plain Coffee, Milk, Sugar $2.7
```

Benefits of Decorator Pattern:

- **Flexibility**: You can add multiple decorators dynamically, each adding new behavior.
- **Single Responsibility Principle**: Each decorator class focuses on a specific feature or responsibility.
- **Avoids Class Explosion**: Instead of creating numerous subclasses to handle all possible combinations of behaviors, decorators allow you to mix and match at runtime.

The decorator pattern is widely used in systems where behavior needs to be extended or modified without changing the original class. For example, it is used in the I/O streams of Java (`BufferedReader`, `InputStreamReader`, etc.).

4o

Powiedziałeś(-aś):
describe and give a code example for Facade

ChatGPT powiedział:
ChatGPT

Facade Pattern

Description:

The **Facade Pattern** provides a simplified interface to a complex subsystem, making it easier for clients to interact with the system. It hides the complexities of the system and provides a more user-friendly interface, while still allowing access to the underlying system if needed. This pattern is particularly useful when working with large and complex libraries or APIs.

Key Concepts:

- **Facade**: The simplified interface that interacts with the subsystem.

- **Subsystem Classes**: These classes perform the actual work but are complex to interact with directly.
- **Client**: The code that interacts with the facade rather than directly with the subsystem.

Example:

Let's consider a **home theater system** that has multiple components like a DVD player, projector, sound system, and lights. Instead of having the client control each of these subsystems individually, we can provide a facade that simplifies the process.

java
Skopiuj kod
```
// Subsystem 1 - DVD Player
class DVDPlayer {
    public void on() {
        System.out.println("DVD Player is on.");
    }

    public void play(String movie) {
        System.out.println("Playing movie: " + movie);
    }

    public void off() {
        System.out.println("DVD Player is off.");
    }
}

// Subsystem 2 - Projector
class Projector {
    public void on() {
        System.out.println("Projector is on.");
    }

    public void off() {
        System.out.println("Projector is off.");
    }

    public void setWideScreenMode() {
        System.out.println("Projector set to wide screen mode.");
    }
}

// Subsystem 3 - Sound System
```

```
class SoundSystem {
    public void on() {
        System.out.println("Sound System is on.");
    }

    public void setSurroundSound() {
        System.out.println("Sound System set to surround sound.");
    }

    public void off() {
        System.out.println("Sound System is off.");
    }
}

// Subsystem 4 - Lights
class Lights {
    public void dim() {
        System.out.println("Lights are dimmed.");
    }

    public void bright() {
        System.out.println("Lights are at full brightness.");
    }
}

// Facade - Home Theater Facade
class HomeTheaterFacade {
    private DVDPlayer dvdPlayer;
    private Projector projector;
    private SoundSystem soundSystem;
    private Lights lights;

    public HomeTheaterFacade(DVDPlayer dvdPlayer, Projector
projector, SoundSystem soundSystem, Lights lights) {
        this.dvdPlayer = dvdPlayer;
        this.projector = projector;
        this.soundSystem = soundSystem;
        this.lights = lights;
    }

    // Simplified interface for watching a movie
```

```
    public void watchMovie(String movie) {
        System.out.println("Get ready to watch a movie...");
        lights.dim();
        projector.on();
        projector.setWideScreenMode();
        soundSystem.on();
        soundSystem.setSurroundSound();
        dvdPlayer.on();
        dvdPlayer.play(movie);
    }

    // Simplified interface for ending a movie
    public void endMovie() {
        System.out.println("Shutting down movie theater...");
        dvdPlayer.off();
        projector.off();
        soundSystem.off();
        lights.bright();
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        // Subsystem components
        DVDPlayer dvdPlayer = new DVDPlayer();
        Projector projector = new Projector();
        SoundSystem soundSystem = new SoundSystem();
        Lights lights = new Lights();

        // Facade
        HomeTheaterFacade homeTheater = new
HomeTheaterFacade(dvdPlayer, projector, soundSystem, lights);

        // Simplified interface for watching a movie
        homeTheater.watchMovie("Inception");

        // Simplified interface for ending a movie
        homeTheater.endMovie();
    }
}
```

Explanation:

1. **Subsystems**: Classes like `DVDPlayer`, `Projector`, `SoundSystem`, and `Lights` are the complex components of the home theater system. Each has its own methods that need to be called in sequence for things to work properly.
2. **Facade**: `HomeTheaterFacade` provides simplified methods (`watchMovie()` and `endMovie()`) that wrap the complex interactions between the subsystems. This allows the client to interact with the system using just a few methods.
3. **Client Code**: The client uses the `HomeTheaterFacade` to control the entire home theater system without needing to know the details of how each subsystem works.

Usage Example Output:

vbnet
Skopiuj kod

```
Get ready to watch a movie...
Lights are dimmed.
Projector is on.
Projector set to wide screen mode.
Sound System is on.
Sound System set to surround sound.
DVD Player is on.
Playing movie: Inception
Shutting down movie theater...
DVD Player is off.
Projector is off.
Sound System is off.
Lights are at full brightness.
```

Benefits of Facade Pattern:

- **Simplified Interface**: It simplifies the use of a complex system by providing a user-friendly interface.
- **Loose Coupling**: It decouples the client from the subsystems, allowing the subsystems to change without affecting the client code.
- **Improved Maintainability**: Complex interactions are encapsulated in a single class, making the system easier to maintain and extend.

The Facade Pattern is commonly used in systems with many interdependent classes, such as graphical frameworks, communication protocols, or libraries that expose low-level details. By using a facade, you can make these systems easier to interact with, improving usability for clients.

4o

Powiedziałeś(-aś):
describe and give a code example for Observer

ChatGPT powiedział:
ChatGPT

Observer Pattern

Description:

The **Observer Pattern** defines a one-to-many relationship between objects, where one object (the **Subject**) notifies multiple dependent objects (**Observers**) of any changes to its state. This pattern is often used to implement distributed event-handling systems where multiple objects need to stay synchronized with a central object.

Key Concepts:

- **Subject**: The object being observed. It maintains a list of observers and notifies them of changes.
- **Observer**: The objects that are dependent on the subject and are notified of any state changes.
- **Concrete Subject**: A subject that changes its state and notifies observers.
- **Concrete Observers**: Objects that subscribe to updates from the subject.

Example:

Let's consider a **weather monitoring system** where the `WeatherStation` (subject) broadcasts weather updates, and different display units (`PhoneDisplay` and `LCDDisplay`, which are observers) update themselves when the weather changes.

java
Skopiuj kod
```
// Observer interface
interface Observer {
    void update(float temperature, float humidity);
}

// Subject interface
interface Subject {
    void registerObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}

// Concrete Subject - WeatherStation
class WeatherStation implements Subject {
    private List<Observer> observers;
```

```
    private float temperature;
    private float humidity;

    public WeatherStation() {
        this.observers = new ArrayList<>();
    }

    // Register an observer
    @Override
    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    // Remove an observer
    @Override
    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    // Notify all observers of state change
    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(temperature, humidity);
        }
    }

    // Set new weather data and notify observers
    public void setWeatherData(float temperature, float humidity) {
        this.temperature = temperature;
        this.humidity = humidity;
        notifyObservers(); // Notify all observers when data changes
    }
}

// Concrete Observer - PhoneDisplay
class PhoneDisplay implements Observer {
    private float temperature;
    private float humidity;

    @Override
```

```
    public void update(float temperature, float humidity) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }

    public void display() {
        System.out.println("Phone Display - Temperature: " +
temperature + "C, Humidity: " + humidity + "%");
    }
}

// Concrete Observer - LCDDisplay
class LCDDisplay implements Observer {
    private float temperature;
    private float humidity;

    @Override
    public void update(float temperature, float humidity) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }

    public void display() {
        System.out.println("LCD Display - Temperature: " +
temperature + "C, Humidity: " + humidity + "%");
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        // Create a weather station (subject)
        WeatherStation weatherStation = new WeatherStation();

        // Create displays (observers)
        PhoneDisplay phoneDisplay = new PhoneDisplay();
        LCDDisplay lcdDisplay = new LCDDisplay();

        // Register observers
```

```
        weatherStation.registerObserver(phoneDisplay);
        weatherStation.registerObserver(lcdDisplay);

        // Simulate weather changes
        weatherStation.setWeatherData(25.5f, 65f);  // Notifies all
observers
        weatherStation.setWeatherData(28.3f, 70f);  // Notifies all
observers again
    }
}
```

Explanation:

1. **Subject Interface**: `Subject` defines methods for managing observers (`registerObserver()`, `removeObserver()`) and notifying them of changes (`notifyObservers()`).
2. **Concrete Subject**: `WeatherStation` implements the `Subject` interface. It maintains a list of observers and notifies them when the weather data changes using `setWeatherData()`.
3. **Observer Interface**: `Observer` defines the method `update()`, which the subject calls to notify observers of changes.
4. **Concrete Observers**: `PhoneDisplay` and `LCDDisplay` implement the `Observer` interface. They receive weather updates and display them through their `display()` methods.
5. **Client Code**: The client creates the `WeatherStation` and the observers (`PhoneDisplay`, `LCDDisplay`), registers them with the subject, and simulates weather data changes.

Usage Example Output:

mathematica
Skopiuj kod

```
Phone Display - Temperature: 25.5C, Humidity: 65.0%
LCD Display - Temperature: 25.5C, Humidity: 65.0%
Phone Display - Temperature: 28.3C, Humidity: 70.0%
LCD Display - Temperature: 28.3C, Humidity: 70.0%
```

Benefits of Observer Pattern:

- **Loose Coupling**: The subject does not need to know the details of the observers, just that they follow a common interface.
- **Dynamic Subscription**: Observers can be added or removed at runtime without affecting the subject.

- **Open for Extension**: New observer types can be added easily without changing existing code.

The Observer pattern is widely used in event-driven systems, GUIs, and in frameworks like Java's `java.util.Observer` or JavaScript's event system. It's a powerful tool for keeping objects in sync when changes occur.